

Hundred days of coding c programming

Day-8

Name = Arayana Khare

Batch = 15

Sap I'd = 590042432

Question no. 1

Write a C program to find the radius, surface area, and volume of a sphere when its diameter is given.

1. Understand the problem statement and identify the required input, output, and formula.

Input:

Diameter of sphere = d Output:

Radius = r

Surface Area = SA

Volume = V Formula:

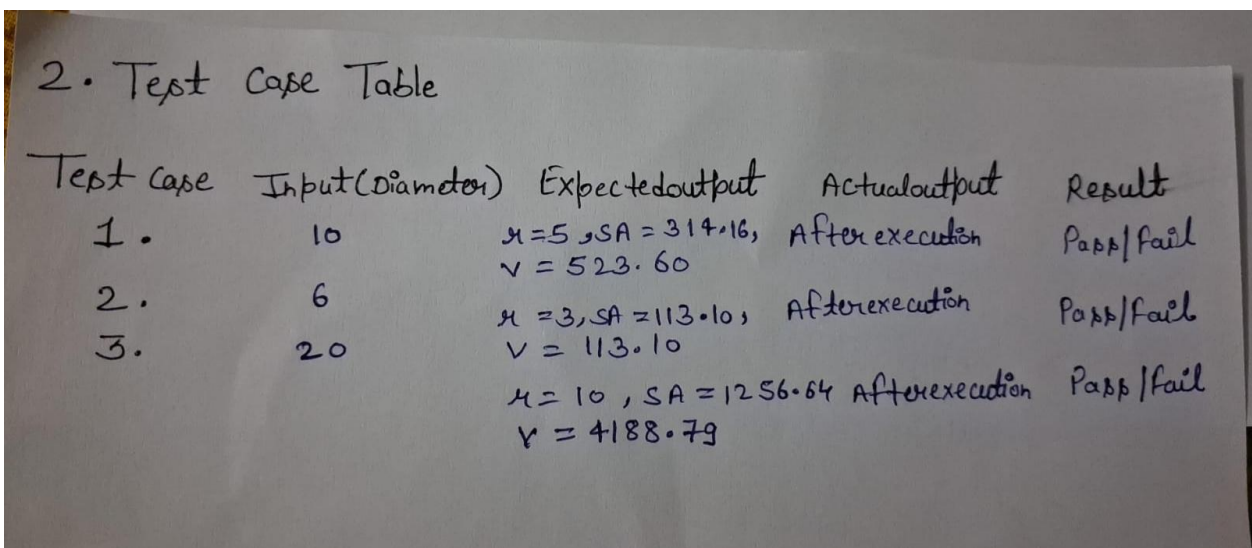
$r = d / 2$

$SA = 4 \times \pi \times r \times r$

$V = (4/3) \times \pi \times r \times r \times r$ Take

$\pi = 3.14159$.

2. Prepare the standard test case table using the following format:



2. Test Case Table

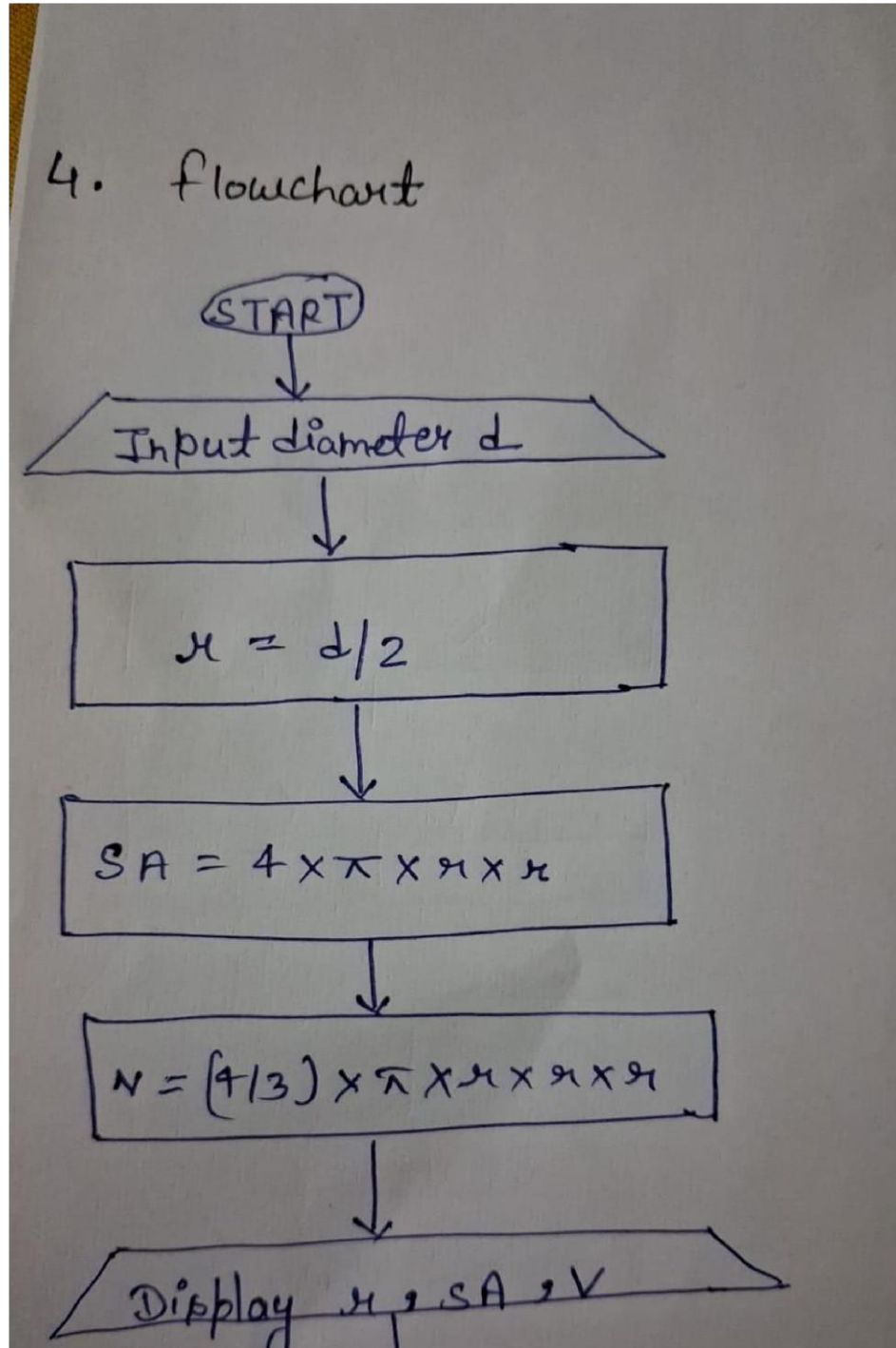
Test Case	Input(Diameter)	Expectedoutput	Actualoutput	Result
1.	10	$r = 5, SA = 314.16, V = 523.60$	After execution	Pass/fail
2.	6	$r = 3, SA = 113.10, V = 113.10$	After execution	Pass/fail
3.	20	$r = 10, SA = 1256.64, V = 4188.79$	After execution	Pass/fail

3. Write (no snippets) and evaluate the algorithm against the test cases prepared in Step 2.

. Algorithm to find radius, surface area and volume of sphere:

1. Start.
 2. Read the diameter d .
 3. Calculate radius using $r = d / 2$.
 4. Calculate surface area using $SA = 4 \times \pi \times r \times r$.
 5. Calculate volume using $V = (4/3) \times \pi \times r \times r \times r$.
 6. Display radius, surface area and volume.
- Stop.

4. Draw (no snippet) and evaluate the flowchart against the test cases prepared in Step 2.



5. Write (no snippets) the C program.

```
ode
#include <stdio.h>

int main()
{
    float d, r, sa, v;
    float pi = 3.14159;

    printf("Enter the diameter of sphere\n");
    scanf("%f", &d);

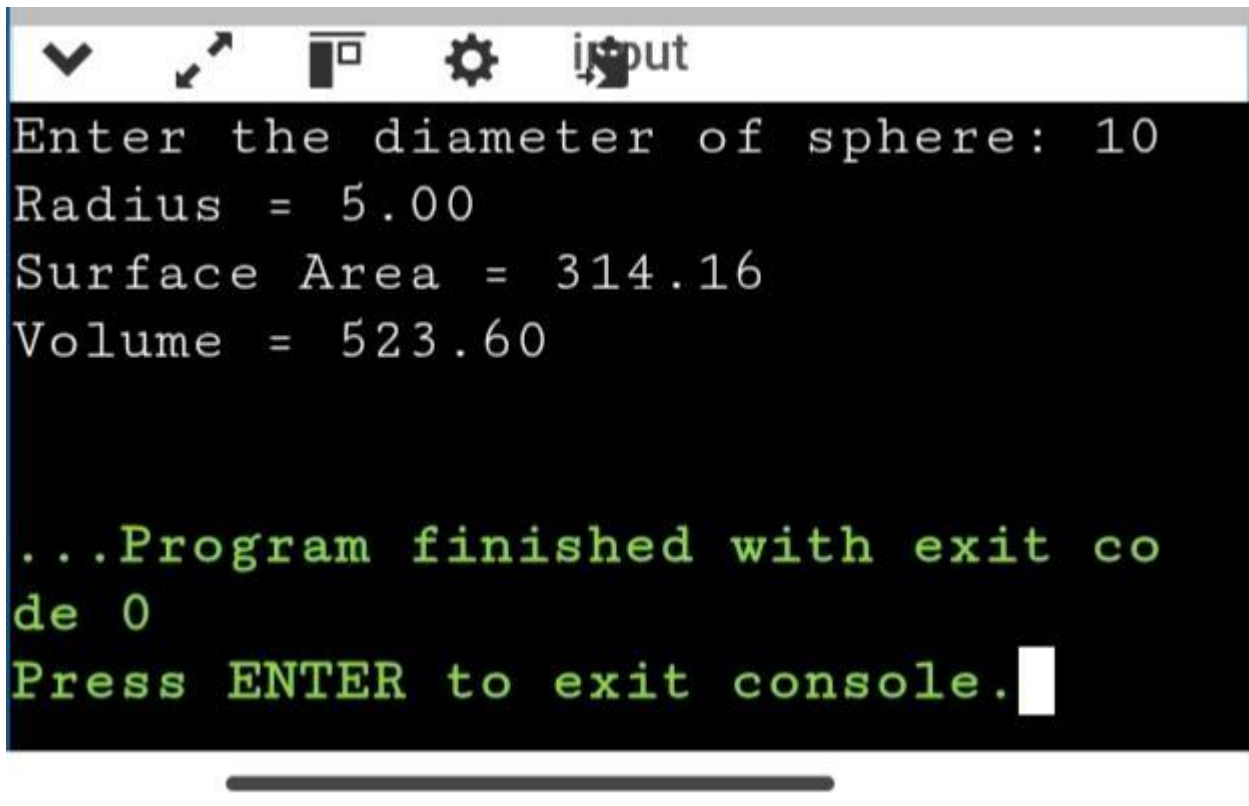
    r = d / 2;
    sa = 4 * pi * r * r;
    v = (4.0 / 3.0) * pi * r * r * r;

    printf("Radius = %.2f\n", r);
    printf("Surface Area = %.2f\n", sa);
    printf("Volume = %.2f\n", v);

    return 0;
}
```

6. Compile and execute the program, then evaluate the code against the test cases prepared in Step 2.

Case 1



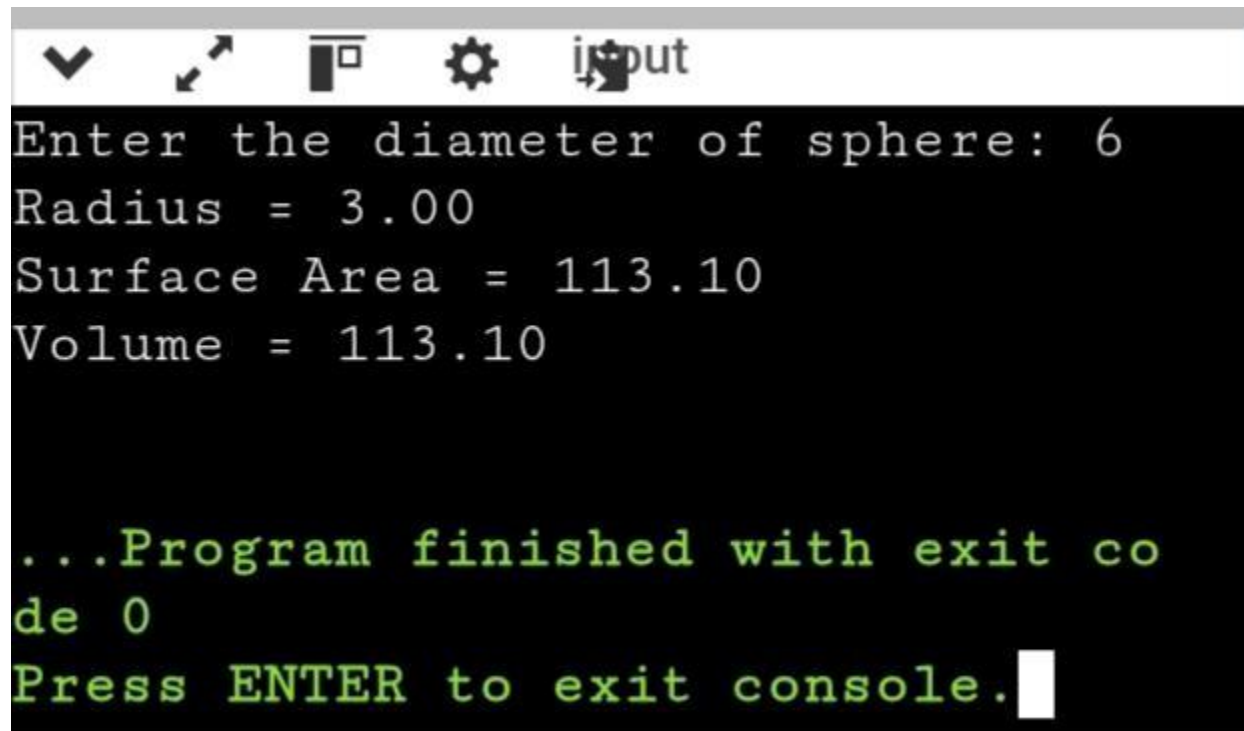
The screenshot shows a console window with a title bar containing icons for window control and a file named 'input'. The console output is as follows:

```
Enter the diameter of sphere: 10
Radius = 5.00
Surface Area = 314.16
Volume = 523.60

...Program finished with exit co
de 0
Press ENTER to exit console.
```

The text is displayed in a monospaced font. The first four lines are in white, and the last three lines are in green. A white cursor is visible at the end of the last line.

Case 2

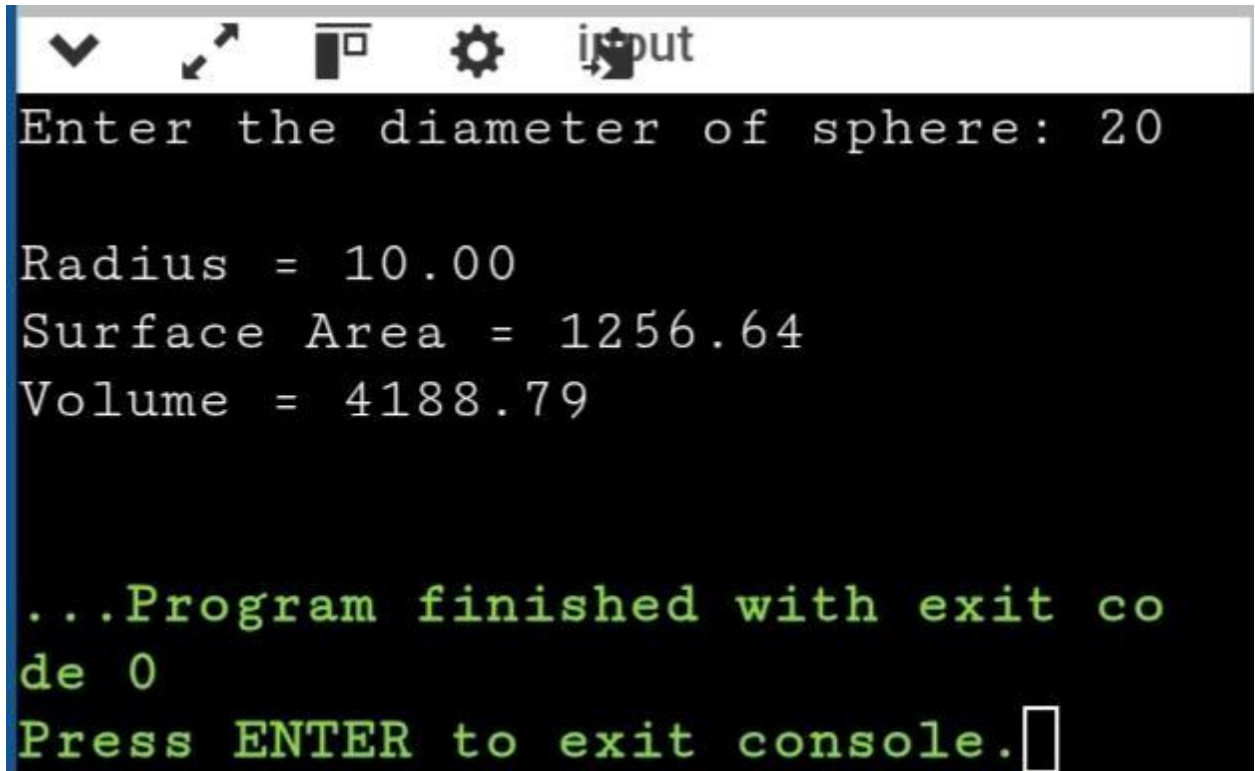


```
Enter the diameter of sphere: 6
Radius = 3.00
Surface Area = 113.10
Volume = 113.10

...Program finished with exit co
de 0
Press ENTER to exit console.
```

The screenshot shows a console window with a title bar containing standard window controls and the text 'input'. The console output displays the results of a program that calculates the radius, surface area, and volume of a sphere given a diameter of 6. The calculations are shown in white text on a black background. The program concludes with a green message indicating it finished with exit code 0 and prompts the user to press ENTER to exit the console.

Case 3



```
Enter the diameter of sphere: 20

Radius = 10.00
Surface Area = 1256.64
Volume = 4188.79

...Program finished with exit co
de 0
Press ENTER to exit console.
```

The screenshot shows a console window with a title bar containing icons for window control and a label 'input'. The main area has a black background with white text for input and output. The output is in green. The program calculates the radius, surface area, and volume of a sphere with a diameter of 20. It ends with a message to press ENTER to exit the console.

7. Attach snippets/screenshots of compilation and execution.

Compilation and Execution

CASE 1

```
#include <stdio.h>

int main()
{
    float d, r, sa, v;
    float pi = 3.14159;

    printf("Enter the diameter of sphere\n");
    scanf("%f", &d);

    r = d / 2;
    sa = 4 * pi * r * r;
    v = (4.0 / 3.0) * pi * r * r * r;

    printf("Radius = %.2f\n", r);
    printf("Surface Area = %.2f\n", sa);
    printf("Volume = %.2f\n", v);

    return 0;
}
```


CASE2

```
#include <stdio.h>

int main()
{
    float d, r, sa, v;
    float pi = 3.14159;

    printf("Enter the diameter of sphere\n");
    scanf("%f", &d);

    r = d / 2;
    sa = 4 * pi * r * r;
    v = (4.0 / 3.0) * pi * r * r * r;

    printf("Radius = %.2f\n", r);
    printf("Surface Area = %.2f\n", sa);
    printf("Volume = %.2f\n", v);

    return 0;
}
```

CASE 3

```
#include <stdio.h>

int main()
{
    float d, r, sa, v;
    float pi = 3.14159;

    printf("Enter the diameter of sphere\n");
    scanf("%f", &d);

    r = d / 2;
    sa = 4 * pi * r * r;
    v = (4.0 / 3.0) * pi * r * r * r;

    printf("Radius = %.2f\n", r);
    printf("Surface Area = %.2f\n", sa);
    printf("Volume = %.2f\n", v);

    return 0;
}
```

Conclusion

The C program successfully calculates the radius, surface area, and volume of a sphere when its diameter is given. The program was compiled and executed successfully, and the output matched the expected results for the test cases.

Question 2

Write a C program to find the image of a point after reflection in the y-axis.

1. Understand the problem statement and identify the required input, output, and formula.

Understand Problem

Input:

Coordinates of point (x, y) Output:

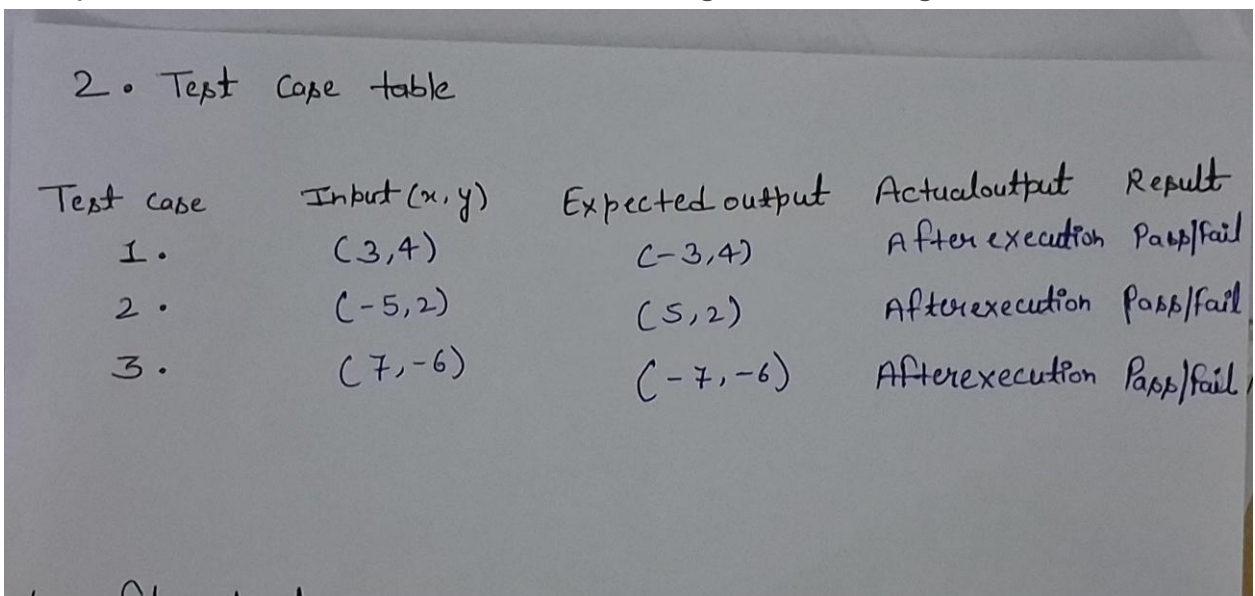
Coordinates of reflected point (x', y') Formula
for reflection in y-axis:

$$x' = -x \quad y' = y$$

Therefore,

Reflected point = (-x, y)

2. Prepare the standard test case table using the following format:



2. Test case table

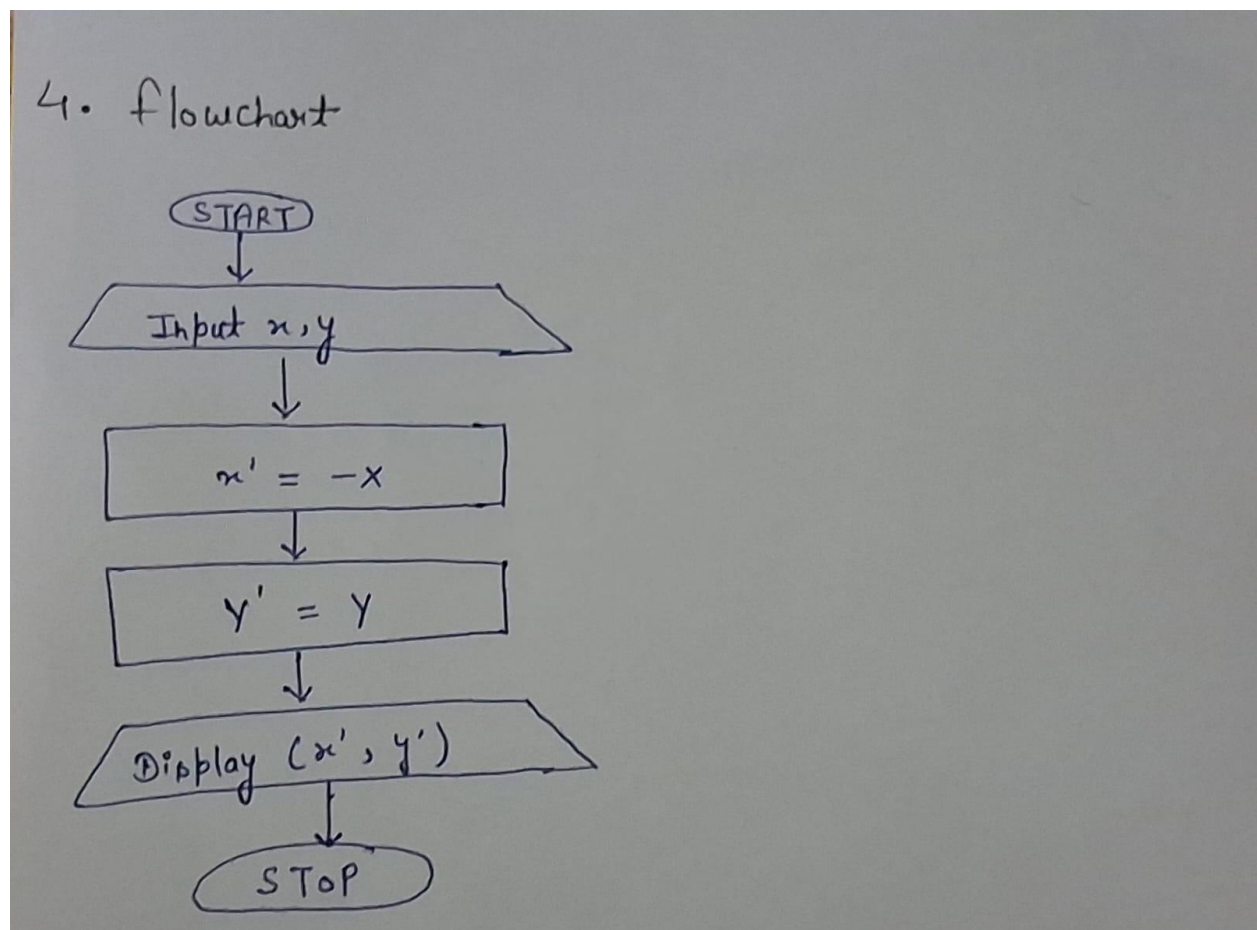
Test case	Input (x, y)	Expected output	Actual output	Result
1.	(3, 4)	(-3, 4)	After execution	Pass/fail
2.	(-5, 2)	(5, 2)	After execution	Pass/fail
3.	(7, -6)	(-7, -6)	After execution	Pass/fail

3. Write (no snippets) and evaluate the algorithm against the test cases prepared in Step 2.

Algorithm

1. Start.
2. Read the coordinates x and y .
3. Calculate $x' = -x$.
4. Keep $y' = y$.
5. Display the reflected point (x', y') .
6. Stop.

4. Draw (no snippet) and evaluate the flowchart against the test cases prepared in Step 2.

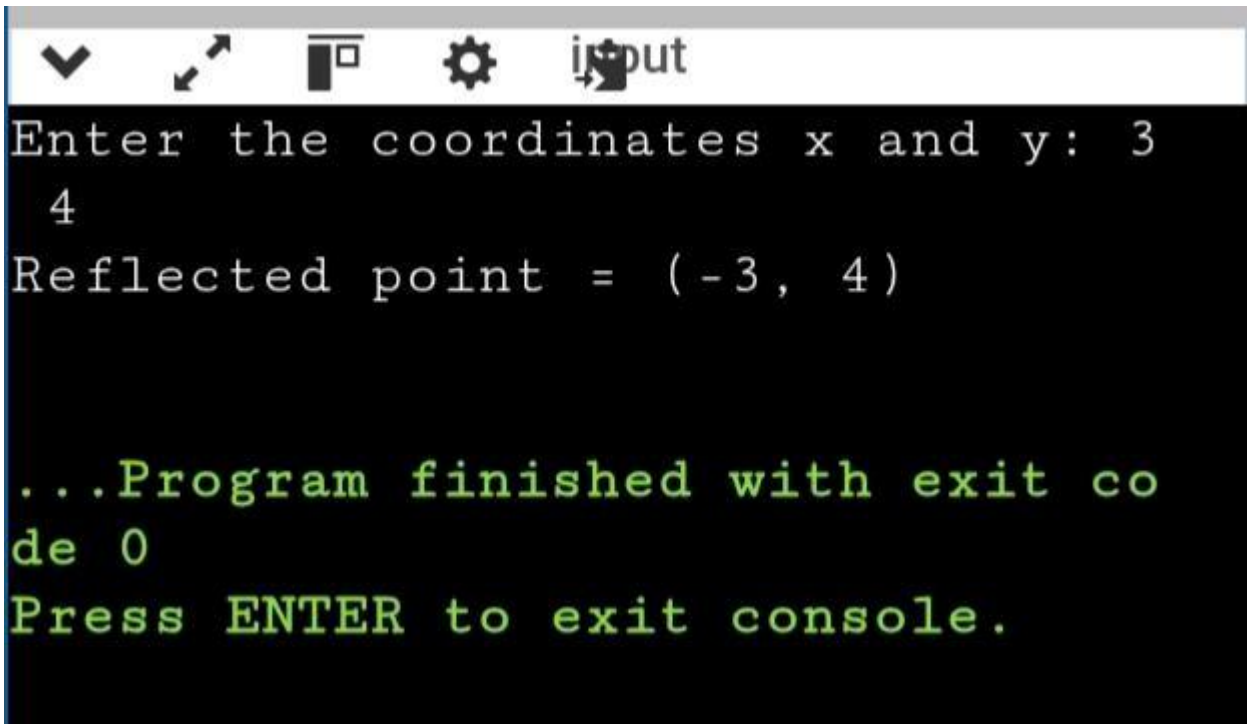


5. Write (no snippets) the C program.

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int x, y, x1, y1;
6
7      printf("Enter the coordinates x and y: ");
8      scanf("%d %d", &x, &y);
9
10     x1 = -x;
11     y1 = y;
12
13     printf("Reflected point = (%d, %d)\n", x1, y1);
14
15     return 0;
16 }
```

6. Compile and execute the program, then evaluate the code against the test cases prepared in Step 2.

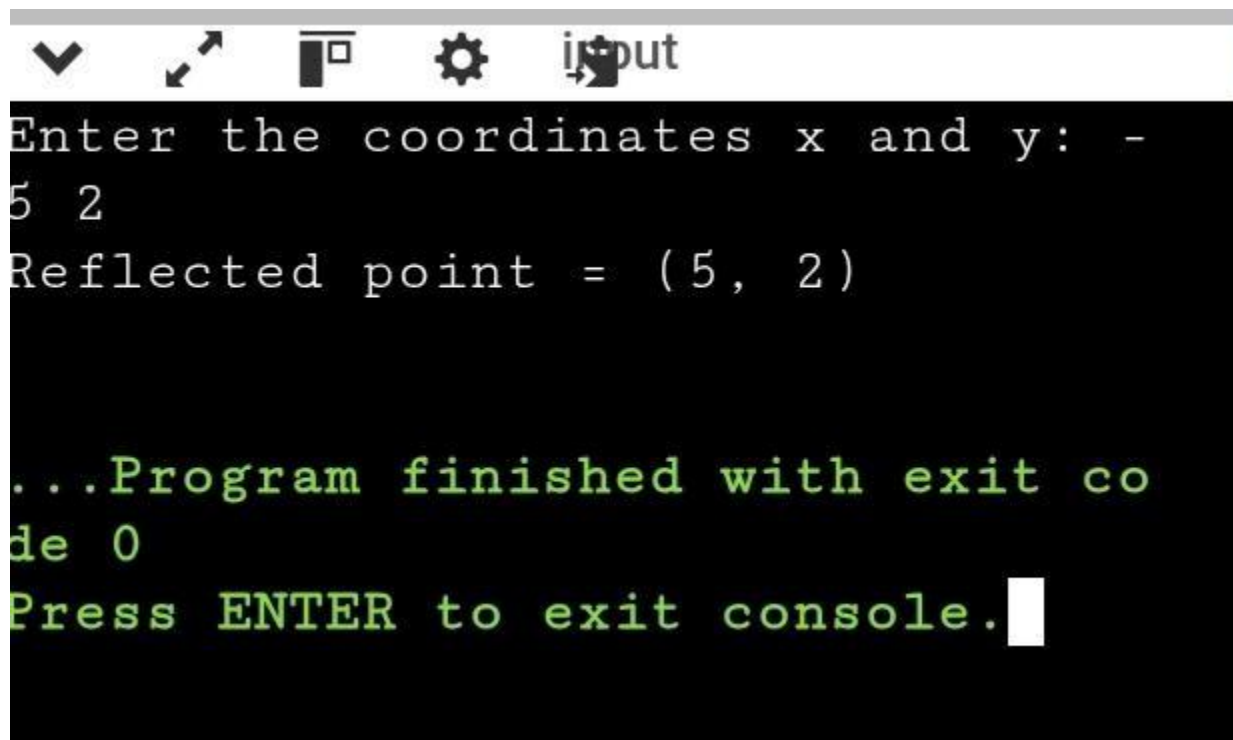
Case 1



```
▼ ↗ □ ⚙ input
Enter the coordinates x and y: 3
4
Reflected point = (-3, 4)

...Program finished with exit co
de 0
Press ENTER to exit console.
```

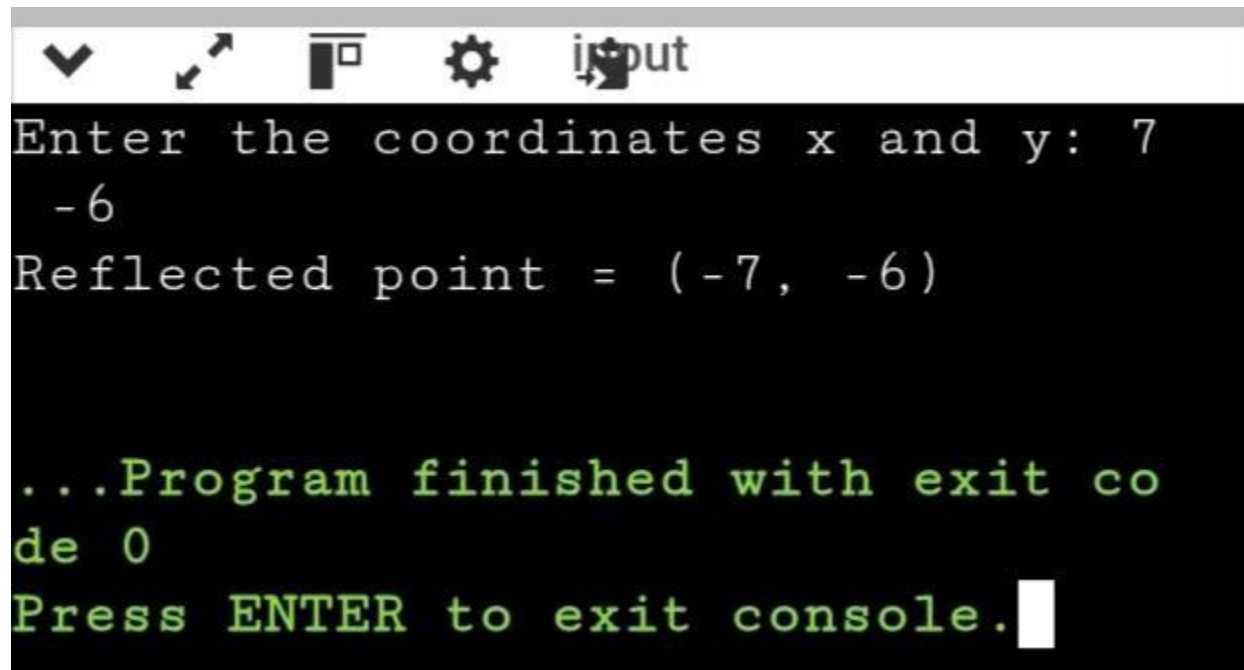
Case 2



```
▼ ↗ □ ⚙ input
Enter the coordinates x and y: -
5 2
Reflected point = (5, 2)

...Program finished with exit co
de 0
Press ENTER to exit console. █
```

Case 3



```
input
Enter the coordinates x and y: 7
-6
Reflected point = (-7, -6)

...Program finished with exit co
de 0
Press ENTER to exit console.█
```


7. Attach Snippets/Screenshot of completion and execution.

Compilation and Execution

CASE 1

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int x, y, x1, y1;
6
7      printf("Enter the coordinates x and y: ");
8      scanf("%d %d", &x, &y);
9
10     x1 = -x;
11     y1 = y;
12
13     printf("Reflected point = (%d, %d)\n", x1, y1);
14
15     return 0;
16 }
```

Enter the coordinates x and y: 3,4
Reflected point = (-3, 1274077648)

...Program finished with exit code 0
Press ENTER to exit console.

CASE 2

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int x, y, x1, y1;
6
7      printf("Enter the coordinates x and y: ");
8      scanf("%d %d", &x, &y);
9
10     x1 = -x;
11     y1 = y;
12
13     printf("Reflected point = (%d, %d)\n", x1, y1);
14
15     return 0;
16 }
```

Enter the coordinates x and y: -5,2
Reflected point = (5, -490303680)

...Program finished with exit code 0
Press ENTER to exit console.

CASE 3

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int x, y, x1, y1;
6
7      printf("Enter the coordinates x and y: ");
8      scanf("%d %d", &x, &y);
9
10     x1 = -x;
11     y1 = y;
12
13     printf("Reflected point = (%d, %d)\n", x1, y1);
14
15     return 0;
16 }
```

Enter the coordinates x and y: 7,-6
Reflected point = (-7, 18420560)

...Program finished with exit code 0
Press ENTER to exit console.

Conclusion

The C program successfully finds the image of a point after reflection in the y-axis. The program correctly changes the sign of the x-coordinate while keeping the y-coordinate unchanged. The output matched the expected results for the test cases.